01-Classical Classifier Baseline

In this notebook, we will do a simple baseline classification using classicial method (Tfidf and logistic regression)

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import precision_recall_fscore_support, f1_score, classificati
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
```

In [2]:
```
# load train and dev set
# train_fname = "../../data/pitt/pitt_train_txt.csv"
# dev_fname = "../../data/pitt/pitt_dev_txt.csv"

train_fname = "../../data/splits/train_50pc.csv"
dev_fname = "../../data/splits/dev_50pc.csv"

train_df = pd.read_csv(train_fname)
dev_df = pd.read_csv(dev_fname)

print("Train len =", len(train_df))
print("Dev df =", len(dev_df))
```

```
Train len = 83
Dev df = 83
```

In [3]:
```
train_df.head()
```

Out[3]:

	file_id	text	target
0	adrso021	Everything you see going on in this picture, g...	CN
1	adrso265	Now you just look at the picture. Okay, this i...	CN
2	adrso237	Tell me everything that you see going on in th...	AD
3	adrso197	Girls and boys getting her one that stores sto...	AD
4	adrso017	All right, we have a few more tasks to go thro...	CN

In [4]:
```
dev_df.head()
```

Out[4]:

	file_id	text	target
0	adrso025	Tell me everything that's going on\n\n\n\nthe ...	AD
1	adrso028	She will find her mother worship trying to dis...	AD
2	adrso032	What's going on well the children are climbing...	AD
3	adrso035	Okay here's the picture tell me I'll tell you ...	AD
4	adrso036	Tell me what's going on\n\n\n\nall the action ...	AD

Pre-processing

We will lowercase the text for better efficiency

In [5]:
```
train_df["text_lower"] = train_df["text"].apply(lambda text: text.lower().strip())
dev_df["text_lower"] = dev_df["text"].apply(lambda text: text.lower().strip())
```

In [6]:
```
train_df.head()
```

Out[6]:

	file_id	text	target	text_lower
0	adrso021	Everything you see going on in this picture, g...	CN	everything you see going on in this picture, g...
1	adrso265	Now you just look at the picture. Okay, this i...	CN	now you just look at the picture. okay, this i...
2	adrso237	Tell me everything that you see going on in th...	AD	tell me everything that you see going on in th...
3	adrso197	Girls and boys getting her one that stores sto...	AD	girls and boys getting her one that stores sto...
4	adrso017	All right, we have a few more tasks to go thro...	CN	all right, we have a few more tasks to go thro...

In [7]:
```
dev_df.head()
```

Out[7]:

	file_id	text	target	text_lower
0	adrso025	Tell me everything that's going on\n\n\n\nthe ...	AD	tell me everything that's going on\n\n\n\nthe ...
1	adrso028	She will find her mother worship trying to dis...	AD	she will find her mother worship trying to dis...
2	adrso032	What's going on well the children are climbing...	AD	what's going on well the children are climbing...
3	adrso035	Okay here's the picture tell me I'll tell you ...	AD	okay here's the picture tell me i'll tell you ...
4	adrso036	Tell me what's going on\n\n\n\nall the action ...	AD	tell me what's going on\n\n\n\nall the action ...

Build Tfidf Vectorizer

We can use Tfidf Vectorizer to convert text to a fixed-length vector

In [8]:
```
vectorizer = TfidfVectorizer()


corpus = train_df["text_lower"]
print(len(corpus))
print("Building Tfidf Vectorizer...")
X_train = vectorizer.fit_transform(corpus)
print("Completed")
```

```
83
Building Tfidf Vectorizer...
Completed
```

In [9]:
```
print(X_train.shape)
```

```
(83, 949)
```

Train Logistic regression using Tfidf vectorizer

We can now train a simple logistic regression classifier using processed X_train

In [10]:
```
X_train = vectorizer.fit_transform(train_df["text_lower"])
y_train = train_df["target"]

print("Training logistic classifier...")
clf = LogisticRegression(random_state=42,
                         class_weight="balanced",
                        ).fit(X_train, y_train)
print("Completed")
```

```
Training logistic classifier...
Completed
```

In [11]:
```
# getting score on train set
train_score = clf.score(X_train, y_train)
print(f"Train accuracy = {train_score*100:.2f} %")
```

```
Train accuracy = 93.98 %
```

Get accuracy using cross validation

In [12]:
```
print("Performing cross-validation...")
scores = cross_val_score(clf, X_train, y_train, cv=5)
print("Completed")
print(scores)
print("Average =", np.average(scores))
```

```
Performing cross-validation...
Completed
[0.52941176 0.76470588 0.76470588 0.6875     0.75       ]
Average = 0.6992647058823529
```

In [13]:
```
# get precision / recall / f1
y_pred = clf.predict(X_train)
print(classification_report(y_train, y_pred, target_names=["AD", "CN"]))
```

```
              precision    recall  f1-score   support

          AD       0.98      0.91      0.95        47
          CN       0.90      0.97      0.93        36

    accuracy                           0.94        83
   macro avg       0.94      0.94      0.94        83
weighted avg       0.94      0.94      0.94        83
```

Now we can evaluate on dev set

In [14]:
```
X_dev = vectorizer.transform(dev_df["text_lower"])
y_dev = dev_df["target"]
print("X_dev shape =", X_dev.shape)
print("y_dev len =", len(y_dev))
```

```
X_dev shape = (83, 949)
y_dev len = 83
```

In [15]:
```
dev_score = clf.score(X_dev, y_dev)
print(f"Dev accuracy = {dev_score*100:.2f} %")
```

```
Dev accuracy = 79.52 %
```

In [16]:
```
y_pred = clf.predict(X_dev)
print(classification_report(y_dev, y_pred, target_names=["AD", "CN"]))
```

```
                precision    recall  f1-score   support

          AD       0.83      0.72      0.77        40
          CN       0.77      0.86      0.81        43

    accuracy                           0.80        83
   macro avg       0.80      0.79      0.79        83
weighted avg       0.80      0.80      0.79        83
```

In [17]:
```
### Process prediction for test set
```

In [18]:
```
# test_fname = "../../data/pitt/pitt_test_txt.csv"
test_fname = "../../data/splits/test_text.csv"

test_df = pd.read_csv(test_fname)
print("Test df =", len(test_df))
test_df["text_lower"] = test_df["text"].apply(lambda text: text.lower().strip())
test_df.head()
```

```
Test df = 71
```

Out[18]:

	ID	Prediction	text	text_lower
0	adrsdt15	NaN	I see oh boy, almost falling on a stool trying...	i see oh boy, almost falling on a stool trying...
1	adrsdt40	NaN	Well the kids are trying to get the cookies an...	well the kids are trying to get the cookies an...
2	adrsdt26	NaN	Okay, this is a family setting in a kitchen an...	okay, this is a family setting in a kitchen an...
3	adrsdt67	NaN	Oh, what do I see in this picture? This remind...	oh, what do i see in this picture? this remind...
4	adrsdt58	NaN	While the young boy is reaching for a cookie j...	while the young boy is reaching for a cookie j...

In [19]:
```
X_test = vectorizer.transform(test_df["text_lower"])
test_df["Prediction"] = clf.predict(X_test)
test_df.head()
```

Out[19]:

	ID	Prediction	text	text_lower
0	adrsdt15	CN	I see oh boy, almost falling on a stool trying...	i see oh boy, almost falling on a stool trying...
1	adrsdt40	CN	Well the kids are trying to get the cookies an...	well the kids are trying to get the cookies an...
2	adrsdt26	CN	Okay, this is a family setting in a kitchen an...	okay, this is a family setting in a kitchen an...
3	adrsdt67	CN	Oh, what do I see in this picture? This remind...	oh, what do i see in this picture? this remind...
4	adrsdt58	CN	While the young boy is reaching for a cookie j...	while the young boy is reaching for a cookie j...

In [20]:
```
# convert to 0/1 to be consistent to ground truth
label2binary = lambda label: "0" if label=="0" or label.lower().strip() == "cn" els
assert label2binary("CN") == "0"
assert label2binary("AD") == "1"
assert label2binary("0") == "0"
assert label2binary("1") == "1"

print("Passed")
```

```
Passed
```

In [21]:
```
# apply to prediction column
test_df["Prediction"] = test_df["Prediction"].apply(label2binary)
# write to prediction output file
output_path = "../../evaluation/"
fname = "test_results_task1_classical.csv"
output_df = test_df.drop(columns=["text", "text_lower"])
output_df.head()
```

Out[21]:

	ID	Prediction
0	adrsdt15	0
1	adrsdt40	0
2	adrsdt26	0
3	adrsdt67	0
4	adrsdt58	0

In [22]:
```
output_df.to_csv(output_path + fname,
                 index=False
                )
print("Completed")
```

```
Completed
```

Output from evaluation scripts

$ python evaluate.py task1_gold.csv test_results_task1_classical.csv Loading data from task1_gold.csv...done Loading data from test_results_task1_classical.csv...done Gold len = 71 Prediction len = 71 Classification report: precision recall f1-score support 0 0.76 0.89 0.82 36 1 0.86 0.71 0.78 35 accuracy 0.80 71 macro avg 0.81 0.80 0.80 71 weighted avg 0.81 0.80 0.80 71 Accuracy = 0.8028169014084507 Recall = 0.7142857142857143

In [ ]: